

An Enhanced Architectural Blueprint for the AI Interviewer Platform

Executive Summary

This document presents a definitive, implementation-ready architectural blueprint for a next-generation, AI-powered platform for conducting and analyzing live technical interviews. It builds upon the foundational microservices vision by specifying a concrete three-tier technology stack designed to optimize for performance, scalability, and developer velocity. The core design philosophy remains centered on a highly resilient, event-driven ecosystem, now enhanced with a rigorous security posture tailored to the unique challenges of Large Language Model (LLM) integration.

The architecture is implemented across three distinct tiers, each leveraging a best-in-class technology stack for its specific domain. The **Presentation Tier** is built with **React**, providing a rich, dynamic user interface and a seamless collaborative coding experience. The **Application Tier**, powered by **Node.js** and the **NestJS** framework, manages all real-time user interactions, session state, and core business logic, excelling at high-concurrency, I/O-bound operations. Finally, the **Intelligence and Execution Tier** utilizes **Python** and the **FastAPI** framework, providing the high-performance, computationally intensive environment required for secure code execution and the sophisticated multi-agent AI analysis system.

A critical and enduring pillar of this architecture is its event-driven nature, with **Apache Kafka** serving as the central nervous system.¹ This strategic choice decouples the real-time Application Tier from the asynchronous Intelligence Tier, ensuring that the user experience remains fluid and responsive, entirely insulated from the performance of backend AI processing. This design not only enhances system resilience but also creates a durable, replayable log of all interview events, forming a rich data substrate for continuous AI model training and future analytical capabilities.

In response to the evolving threat landscape, this blueprint introduces a dedicated and comprehensive security framework guided by the **OWASP Top 10 for LLM Applications**.² It moves beyond traditional application security to address novel AI-specific attack vectors, with a deep focus on mitigating risks such as prompt injection, insecure output handling, and data poisoning. This proactive security-by-design approach is fundamental to building user trust and ensuring the long-term integrity and reliability of the platform's assessments.

This report serves as a single source of truth and an actionable roadmap for the engineering team. It provides a strategic and tactical guide for building a production-grade system that is not only robust and scalable today but is also architected for continuous evolution and

innovation in the rapidly advancing field of AI-driven talent assessment.

The Three-Tier AI Interviewer Architecture

This section translates the conceptual microservices architecture into a concrete, three-tier implementation plan. By explicitly defining the technology stack and responsibilities for each layer, this blueprint provides a clear and actionable structure for development teams. The chosen technologies—React for the frontend, Node.js for the application backend, and Python for AI and execution workloads—represent a strategic decision to leverage the optimal ecosystem for each distinct architectural domain.

High-Level System Diagram and Technology Mapping

The revised architecture segments the platform's services into three logical tiers: Presentation, Application, and Intelligence/Execution. All client traffic is routed through a central API Gateway, which handles authentication and directs synchronous requests to the Node.js Application Tier. The core of the system's resilience and scalability lies in the asynchronous communication channel provided by Apache Kafka, which serves as the boundary between the real-time Application Tier and the data-intensive Intelligence Tier. The following table provides a definitive mapping of the conceptual microservices to the specified technology stack, clarifying ownership and primary responsibilities for each component. This mapping is foundational for team structure, resource allocation, and parallel development efforts.

Microservice (Conceptual)	Architectural Tier	Core Technology	Primary Responsibilities
Gateway Service	Edge	Kong / Ambassador	Authentication (JWT validation), rate limiting, routing to Node.js services.
User & Auth Service	Application	Node.js (NestJS)	User profile management, registration/login, JWT issuance, RBAC. Interacts with Postgres via Prisma.
Interview Orchestration Service	Application	Node.js (NestJS)	Interview lifecycle management (scheduling, state changes). Interacts with Postgres via

			Prisma.
Real-Time Collaboration Service	Application	Node.js (NestJS)	Manages WebSocket connections, synchronizes CRDT state for the code editor via Redis Pub/Sub.
Results & Reporting Service	Application	Node.js (NestJS)	Consumes final analysis results from Kafka, aggregates reports, and persists them to Postgres.
Event Ingestion Service	Intelligence & Execution	Python (FastAPI)	High-throughput endpoint for receiving all client/server events and publishing them reliably to Kafka.
Code Execution Service	Intelligence & Execution	Python (FastAPI)	Executes untrusted code in a secure, containerized sandbox (Docker + gVisor). Publishes results to Kafka.
AI Analysis Service (Multi-Agent)	Intelligence & Execution	Python (FastAPI, LangChain)	Consumes event streams from Kafka, performs multi-modal analysis, and publishes structured insights back to Kafka.

Presentation Tier: The React Frontend

The Presentation Tier is the sole touchpoint for users and is responsible for delivering a responsive, intuitive, and collaborative interview experience.

Core Framework and State Management

The frontend will be a Single-Page Application (SPA) built using **React**. This choice is driven by React's vast ecosystem, component-based architecture, and strong community support,

which accelerates development of complex user interfaces. For managing the application's complex and dynamic state—particularly during a live interview session—**Redux Toolkit** will be employed.⁴ Redux Toolkit is the modern, opinionated standard for Redux development, providing a scalable pattern for centralizing application state. It simplifies store setup, reduces boilerplate code through the use of "slices," and integrates powerful tools for asynchronous logic and data fetching.⁵ This centralized store will hold the interview session state, user information, real-time code editor content, and communication status, ensuring a consistent and predictable data flow throughout the application.

Collaborative Code Editor Integration

A core feature of the platform is the real-time collaborative code editor. To provide a professional, feature-rich experience comparable to modern IDEs, the **Monaco Editor**—the engine that powers VS Code—will be integrated.⁷

Integration will be streamlined using the `@monaco-editor/react` package.⁸ This wrapper simplifies the process of embedding the editor as a React component, automatically handling its complex instantiation and disposal within the React component lifecycle.⁸ Key props such as language, value, theme, and options will be used to configure the editor's appearance and behavior dynamically.

The frontend will manage the editor's state locally and synchronize it with the backend. The `onMount` prop will be used to gain a reference to the editor instance for programmatic control, while the `onChange` event handler will capture user edits.⁹ These edits will be converted into Conflict-Free Replicated Data Type (CRDT) operations and sent to the Node.js backend via the persistent WebSocket connection. Conversely, CRDT operations received from the server will be applied to the local editor instance, ensuring all participants see a consistent document state with minimal latency.¹

Real-Time Communication

Upon entering an interview session, the React client will establish a persistent WebSocket connection with the Node.js Real-Time Collaboration Service. This full-duplex channel is the conduit for all low-latency communication during the live session.¹ All real-time events, including CRDT operations for code edits, requests to execute code, and client-side proctoring events (e.g., browser tab switching), will be encapsulated in a structured JSON format and transmitted over this channel.

Application Tier: The Node.js Backend

The Application Tier serves as the primary backend for handling user-facing business logic, managing persistent data, and orchestrating the real-time aspects of the interview.

Framework Choice: NestJS

NestJS is selected as the Node.js framework for this tier. Unlike more unopinionated frameworks, NestJS provides a highly structured, modular architecture heavily inspired by Angular, which promotes a clear separation of concerns through modules, controllers, and injectable services.¹¹ Its first-class support for TypeScript, combined with this organized structure, makes it exceptionally well-suited for building scalable and maintainable enterprise-grade applications, which aligns perfectly with the defined responsibilities of the application services.¹³

Service Responsibilities

- **Data Persistence and Authentication:** The User & Auth and Interview Orchestration services will be implemented as distinct NestJS modules. For database interaction, **Prisma** will be used as the ORM.¹¹ Prisma's schema-first approach and auto-generated, type-safe client provide a robust and developer-friendly interface to the PostgreSQL database, reducing the likelihood of runtime data-related errors.¹⁵ JWT-based authentication will be managed using the official `@nestjs/jwt` and `@nestjs/passport` packages. A dedicated AuthService will handle user credential validation and the issuance of signed JWTs. A global AuthGuard will then be used to protect sensitive endpoints by validating the JWT on every incoming request, ensuring a secure-by-default posture.¹⁶
- **Real-Time Collaboration:** This service is the heart of the live experience and will be built using NestJS's native WebSocket Gateway (`@nestjs/websockets`). This gateway will manage the lifecycle of all client WebSocket connections. When it receives a CRDT operation from one client, it will not process it directly. Instead, to support horizontal scalability, it will publish the operation to a session-specific channel in **Redis Pub/Sub**. All other NestJS server instances running the collaboration service will be subscribed to these channels. Upon receiving a message from Redis, the instance will then broadcast the CRDT operation to all of its connected clients for that session. This fan-out pattern ensures that state is synchronized consistently across a distributed set of server nodes, a critical requirement for a highly available system.¹

Intelligence & Execution Tier: The Python Backend

This tier is dedicated to the computationally intensive and specialized tasks of the platform:

secure code execution, high-volume event ingestion, and AI-driven analysis.

Framework Choice: FastAPI

FastAPI is the chosen framework for all Python services. Its primary advantages are its exceptional performance, which rivals that of Node.js for I/O-bound tasks, and its native support for asynchronous programming.¹⁸ This makes it an ideal choice for building the high-throughput Event Ingestion Service. Furthermore, its automatic generation of OpenAPI-compliant documentation and its use of Python type hints for data validation simplify API development and ensure correctness, which is crucial for the complex data models used by the AI services.¹⁹

Service Responsibilities

- **Event Ingestion Service:** This will be a lean, high-performance FastAPI endpoint designed for one purpose: to receive event data and reliably publish it to Kafka. It will accept POST requests with batches of events, perform basic schema validation using Pydantic models, and then use an asynchronous, highly optimized Kafka client library (e.g., aiokafka) to produce the messages. Its fully async nature ensures that it can handle a massive volume of incoming events from thousands of concurrent interviews without blocking or becoming a bottleneck.
- **Audio Streaming:** For real-time audio analysis, the client will stream raw audio data (e.g., in Opus format) via a dedicated WebSocket endpoint on a FastAPI server.²¹ The server will be architected to handle binary WebSocket frames, receiving audio chunks and forwarding them directly to the Speech & Veracity Analysis Agent for processing. This direct streaming approach minimizes latency, which is critical for real-time transcription and vocal analysis.²²
- **Secure Code Execution Service:** This service exposes a FastAPI endpoint that accepts a code snippet, language identifier, and any standard input. The core of its implementation is a rigorous sandboxing mechanism. For each execution request, it will dynamically provision an isolated environment using containerization technologies like **Docker**.²⁴ To provide a deeper layer of security and prevent kernel-level exploits, these containers will be run using a sandboxing runtime such as **gVisor**, which intercepts system calls and provides a strong isolation boundary.²⁵ Strict resource limits on CPU time, memory usage, and execution duration will be enforced to prevent denial-of-service attacks. Network access from within the sandbox will be disabled by default. The service captures stdout, stderr, and performance metrics, then packages them into a `code_execution_result` event and publishes it to Kafka for downstream analysis.¹

The architectural seam created by placing Apache Kafka between the Node.js and Python

tiers is the most critical decision for ensuring system resilience and promoting team autonomy. This event-driven approach provides temporal decoupling, meaning the producers of events (the Node.js services managing the live interview) do not need to wait for, or even be aware of, the consumers (the Python AI services).¹ If an AI analysis agent is slow or temporarily unavailable, the user experience in the live interview remains completely unaffected. The Node.js service simply publishes the `code_executed` event to Kafka's durable log and its responsibility ends. The Python services can then consume and process these events at their own pace. This design insulates the critical, real-time user path from the non-critical, asynchronous analytical path, dramatically improving the platform's fault tolerance. It also empowers the AI and application teams to develop, deploy, and scale their services independently, accelerating the overall development lifecycle.

The Intelligence Layer: A Multi-Agent System with LangChain

The platform's primary differentiator is its intelligence layer, which moves beyond a single monolithic model to a collaborative multi-agent system. This architecture allows for specialized, deep analysis of the diverse data streams—code, speech, and language—generated during an interview. By decomposing the analytical problem, we can deploy autonomous agents that excel at specific tasks, working in concert to produce insights that are far richer and more reliable than any single agent could achieve alone.¹

Agent Roles and Collaborative Workflow with LangChain

To build and orchestrate this multi-agent system, the architecture will leverage the **LangChain** framework. LangChain provides a robust set of abstractions and tools for creating agents, equipping them with capabilities, managing their state and memory, and defining the logic for their interaction.⁸ This choice accelerates development by providing the essential building blocks for constructing sophisticated, reasoning-driven AI workflows.²⁶

The system is orchestrated by a central planner agent that delegates tasks to a team of specialized agents, with the entire workflow driven by events consumed from Apache Kafka.

- **The Orchestrator Agent:** This agent functions as the system's "planner" or "supervisor".²⁶ Implemented as a LangChain supervisor agent, its primary role is to consume the raw event stream from the interview-events Kafka topic. It does not perform analysis itself. Instead, it is prompted to decompose the high-level goal of "analyze this interview" into a sequence of concrete sub-tasks. It determines which specialized agent is best suited for each piece of incoming data and routes tasks accordingly. For example, upon receiving a `code_execution_result` event, it will invoke the Code Quality & Logic Agent as a "tool," passing the relevant code payload.²⁶

- **The Code Quality & Logic Agent:** This agent is an expert in software engineering principles. It is triggered by the Orchestrator with data from `code_execution_result` events. Its toolkit, implemented as LangChain tools, includes static analysis linters, code complexity calculators (e.g., for cyclomatic complexity), and performance analysis utilities. Crucially, it also leverages an LLM fine-tuned on a vast corpus of code to assess the problem-solving approach, algorithmic correctness, and the clarity and maintainability of the candidate's solution.¹
- **The Communication & Personality Insights Agent (NLP):** This agent focuses on the *content* of the candidate's speech by analyzing the transcribed text. It employs a suite of Natural Language Processing (NLP) techniques to evaluate communication skills such as clarity, conciseness, and structure.²⁸ Beyond surface-level analysis, it is prompted to infer psychological traits based on established models like the Big Five personality traits (Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism) by analyzing linguistic patterns.⁷ For instance, it analyzes the use of positive versus negative emotional language, linguistic complexity, and pronoun usage, which have been shown to correlate with personality and cognitive states.²⁸
- **The Speech & Veracity Analysis Agent:** This agent analyzes the raw audio stream from the audio-streams topic, focusing on *how* the candidate speaks, not just what they say. It operates on the principle that cognitive load, stress, and deception can manifest in subtle vocal cues.¹⁵ The agent is designed to extract and analyze prosodic features of the voice, such as changes in pitch (vocal tone), speech rate (cadence), volume, and the frequency of speech disfluencies (e.g., "uh," "um").¹² Deviations from a candidate's established baseline vocal pattern can signal moments of increased cognitive effort or uncertainty, providing valuable context to their verbal responses.¹⁵

Implementing Proactive Security with LLM Guardrails

Securing a dynamic, agentic system requires moving beyond traditional perimeter defenses. The architecture will implement security *within* the AI reasoning loop itself by integrating a guardrails framework. Libraries such as **NVIDIA NeMo Guardrails**, **Guardrails AI**, or custom **LangChain Middleware** provide the necessary capabilities to enforce policies on the inputs, outputs, and actions of the LLM agents.²⁹

This approach creates what can be conceptualized as an "internal AI firewall." While traditional security focuses on sanitizing external inputs before they reach the application, an internal firewall inspects and validates the intermediate "thoughts" and actions generated by the AI agents themselves. This is a critical second-order security measure. It ensures that even if one agent is compromised through a sophisticated prompt injection, its ability to perform unauthorized actions or manipulate other agents within the system is severely restricted. This containment of the "blast radius" is essential for securing complex, multi-agent systems.

The guardrails will be implemented in three key areas:

- **Input Guardrails:** Before any data—be it transcribed speech or a candidate's code—is passed to an LLM for analysis, a set of input guards will screen it. These guards will use a combination of rule-based filters (e.g., regex for known attack patterns) and classifier models to detect and block potential prompt injection attempts or malicious content before it can influence the agent's behavior.³²
- **Output Guardrails:** After an agent's LLM generates a response but before that response is used for any downstream action (like being published as an insight or calling a tool), output guards will validate it. These checks ensure the output conforms to a predefined structure, does not contain harmful content (like executable scripts), and does not attempt to leak sensitive information from its prompt.³³
- **Procedural Guardrails (Tool Access Control):** This is the most critical layer. The ability of any agent to invoke tools—whether they are external functions like a linter or internal calls to other agents—will be strictly governed by policies. This enforces the **Principle of Least Privilege** at the agent level.³⁵ For example, the Communication Insights Agent, which only needs to process text, will be explicitly forbidden from accessing any tools related to code execution or network access. This is a direct and powerful defense against the "Excessive Agency" (LLM06) vulnerability, where an LLM performs actions it should not be allowed to.²

Security Posture: Defending Against Advanced LLM Attacks

The integration of Large Language Models introduces a new class of vulnerabilities that traditional security measures are not equipped to handle. This section establishes a comprehensive security posture dedicated to these unique threats, using the **OWASP Top 10 for LLM Applications** as a guiding framework. This proactive approach moves beyond general application security to address specific, modern attack vectors, ensuring the platform is resilient against manipulation and misuse.

The Modern LLM Threat Landscape (OWASP Top 10)

The OWASP Top 10 for LLM Applications provides an industry-standard, consensus-driven framework for understanding and mitigating the most critical security risks in AI systems.² Adopting this framework ensures that our security strategy is comprehensive and aligned with current best practices. The ten key vulnerabilities identified by OWASP are:

1. Prompt Injection (LLM01)
2. Sensitive Information Disclosure (LLM02)
3. Supply Chain Vulnerabilities (LLM03)
4. Data and Model Poisoning (LLM04)

5. Improper Output Handling (LLM05)
6. Excessive Agency (LLM06)
7. System Prompt Leakage (LLM07)
8. Vector and Embedding Weaknesses (LLM08)
9. Misinformation (LLM09)
10. Unbounded Consumption (LLM10)

This section will perform a deep dive into the three vulnerabilities most critical to the AI Interviewer platform: Prompt Injection, Improper Output Handling, and Data Poisoning.

Deep Dive: Mitigating Prompt Injection (LLM01)

Prompt injection is the most prevalent and critical vulnerability affecting LLM applications.² It occurs when an attacker manipulates the model's input to cause unintended behavior, such as bypassing safety controls or executing unauthorized commands.³⁶

Threat Analysis

The primary attack vectors for prompt injection include:

- **Direct Injection (Jailbreaking):** This involves an attacker directly embedding malicious instructions in their input to override the LLM's original system prompt. Common examples include commands like "Ignore all previous instructions and..." or using role-playing scenarios (e.g., the "DAN" or "Grandmother" tricks) to coax the model into violating its safety policies.³⁶
- **Indirect Injection:** This more subtle attack occurs when the LLM ingests and processes malicious instructions from a compromised external data source. For this platform, a significant risk is a candidate submitting code that contains malicious instructions hidden within comments. When the Code Quality Agent analyzes this code, it could inadvertently execute the hidden prompt.³⁷
- **Obfuscation Techniques:** Attackers can bypass simple keyword-based filters by obfuscating their malicious prompts. This includes using different encodings (e.g., Base64, Hex), exploiting typoglycemia (e.g., "ignroe all prevoius instructions"), or hiding instructions in non-textual formats like images or specially formatted Markdown that are processed by multimodal models.³⁶

Defense-in-Depth Strategy

Because no single technique can fully prevent prompt injection, a multi-layered defense-in-depth strategy is essential.⁴⁰ This approach combines several complementary controls to create a robust and resilient security posture.

Layer	Technique	Implementation Details
1. Input Filtering	Pre-processing and sanitization of all inputs before they reach the LLM.	A dedicated input validation layer will be implemented. This can use a combination of rule-based filters to detect known attack patterns (e.g., "ignore previous instructions") and a separate, simpler classifier model trained to identify potential injection attempts. All external content, such as code from the editor, will be sanitized to strip or neutralize potentially executable comments or metadata before analysis. ³⁵
2. Prompt Engineering	Structuring prompts to create a clear separation between trusted instructions and untrusted data.	All system prompts for the AI agents will be engineered to use strong delimiters to clearly demarcate the boundary between the trusted system instructions and the untrusted user-provided data (e.g., ### INSTRUCTIONS ###... ### USER DATA ###). The prompts will also include explicit instructions for the model to treat any content in the user data section as inert text to be analyzed, not as commands to be executed. ³⁵
3. Output Monitoring	Validating the LLM's generated output before it is used by any downstream system.	After an LLM generates a response but before it is acted upon (e.g., used to call a tool), an output validation step will check its content. This includes ensuring the output conforms to the expected format (e.g., valid JSON), scanning for malicious code snippets, and checking for

		attempts to leak parts of the system prompt. This is a critical control for preventing Insecure Output Handling (LLM05). ³⁶
4. Privilege Control	Applying the Principle of Least Privilege to the LLM agent's tools and capabilities.	This is one of the most effective defenses. The LLM agents will not have direct access to sensitive resources like databases or external APIs. All actions will be mediated through narrowly-scoped, well-defined tools. For example, the Code Quality Agent can call a "run_linter" tool but cannot call a "send_email" tool. This severely limits the potential damage from a successful injection, as the compromised agent can only perform actions within its limited set of permissions. ³⁵
5. Human-in-the-Loop	Requiring human approval for high-risk or irreversible actions.	For any actions deemed critical, such as finalizing a candidate's score or sending a report, the system will require an explicit confirmation step from an authorized human user (i.e., the interviewer). This ensures that a fully autonomous, and potentially compromised, agent cannot take high-stakes actions without oversight. ³⁷

Deep Dive: Preventing Improper Output Handling (LLM05)

This vulnerability arises from the flawed assumption that output from an LLM can be trusted by default.² An attacker can leverage a prompt injection attack to cause the LLM to generate a malicious payload, which is then passed to and executed by a downstream component of the

application.⁴⁴ For example, an attacker could trick the LLM into generating a JavaScript payload in its summary, which, if rendered directly in a browser, would lead to a Cross-Site Scripting (XSS) attack.⁴²

Mitigation Strategy

- **Treat LLM Output as Untrusted User Input:** This is the foundational principle for mitigating this vulnerability. All content generated by an LLM must be subjected to the same rigorous validation, sanitization, and encoding procedures as input coming directly from an external user.³⁹
- **Context-Aware Output Encoding:** Before rendering or using any LLM-generated content, it must be encoded for the specific context in which it will be used. For example, if displaying an LLM's text analysis in a web report, the text must be HTML-encoded to neutralize any potential script tags. If an LLM generates data to be used in a database query, that data must be passed through parameterized queries, never concatenated directly into a SQL string.⁴²
- **Sandboxing of Executable Output:** In any scenario where the LLM is intended to generate executable content—such as the candidate's code being passed to the Code Execution Service—that content must be run within a strictly isolated and resource-constrained sandbox environment. This ensures that even if the code is malicious, it cannot harm the host system or other parts of the platform.⁴⁴

Deep Dive: Securing the Data Pipeline Against Poisoning (LLM04)

Data poisoning is an integrity attack where an attacker deliberately corrupts the data used for model training, fine-tuning, or Retrieval-Augmented Generation (RAG) to introduce biases, vulnerabilities, or hidden backdoors.² A successful attack could cause the model to generate factually incorrect analyses, exhibit biased behavior, or even execute a hidden function when a specific trigger phrase is encountered.⁴⁸

Mitigation Strategy

- **Data Provenance and Verification:** The most critical defense is to maintain strict control over the data supply chain. All data sources used for training, fine-tuning, or populating the knowledge base for RAG must be rigorously vetted for trustworthiness. A clear data lineage must be maintained, tracking the origin of all data and any transformations applied to it. Tools like Machine Learning Bill of Materials (ML-BOM) can be used to formalize this process.⁴⁸
- **Sanitization on Ingestion:** When new data is added to the system (e.g., a new

technical document for the RAG vector database), it must be scanned for malicious content, including hidden prompts or toxic language, before it is processed and embedded.⁴⁹

- **Anomaly Detection in Training:** During any fine-tuning process, key training metrics such as loss and accuracy should be closely monitored. Unexpected spikes or deviations from normal patterns can be an indicator of a data poisoning attempt and should trigger an immediate investigation.⁴⁸
- **Data Segregation:** User-supplied data from one interview session must never be used to directly influence the analysis of another. Each interview's context and data should be logically and, where possible, physically segregated to prevent cross-contamination or the inadvertent training of models on potentially private or malicious user input.⁴⁸

The System Contract: Formal API Definitions

To ensure clear, consistent, and predictable interactions between the platform's distributed components, this section provides a formal technical specification for all Application Programming Interfaces (APIs). These definitions serve as a binding contract between the React frontend, the Node.js application services, and the Python intelligence services. Adhering to these contracts is essential for enabling parallel development, simplifying integration, and facilitating automated testing.

The formalization of these contracts transforms the architectural design into an executable engineering plan. By providing machine-readable specifications like an OpenAPI document, teams can work in parallel against a mocked API server generated directly from the spec. This allows the frontend team to build UI components that consume an API before the backend implementation is complete. Furthermore, these contracts enable automated contract testing within the CI/CD pipeline, which can programmatically verify that changes in a service do not break the expectations of its consumers, thereby reducing integration friction and improving overall development velocity.

API Design Principles: REST and WebSocket

The platform employs two primary API styles, each selected to align with the nature of the interaction it facilitates.

- **RESTful APIs:** All synchronous, request-response operations are exposed via a RESTful API over HTTP. This includes standard CRUD (Create, Read, Update, Delete) actions such as user authentication, scheduling interviews, and fetching completed reports. The API design strictly adheres to modern REST conventions, including⁵²:
 - **Resource-Oriented URLs:** Endpoints are structured around logical resources (e.g., /users, /interviews).
 - **Semantic HTTP Verbs:** HTTP methods are used according to their defined

- semantics (GET for retrieval, POST for creation, PUT/PATCH for updates, DELETE for removal).
 - **Statelessness:** Each request from a client contains all the information necessary for the server to process it, without relying on server-side session state.
 - **JSON Payloads:** All request and response bodies utilize the JSON data format.
 - **Standard HTTP Status Codes:** Conventional codes are used to indicate the outcome of an API call (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found).
- **WebSocket APIs:** For all real-time, low-latency, and bidirectional communication required during a live interview, the platform utilizes the WebSocket protocol.⁵³ A single, persistent WebSocket connection is established between the client and the Real-Time Collaboration Service. This full-duplex channel is used to stream CRDT operations for the code editor, send code execution requests, receive execution results, and transmit proctoring events.⁵⁵ All communication over this channel uses a structured JSON message format to ensure clarity and consistency.

REST API Endpoints (OpenAPI 3.0 Specification)

The following provides a formal OpenAPI 3.0 specification for the key RESTful resources. This specification serves as the single source of truth for the platform's synchronous APIs. Best practices such as clear operationIds, detailed descriptions, and reusable schema components are employed to enhance clarity and maintainability.⁵⁶

YAML

```

openapi: 3.0.3
info:
  title: AI Interviewer Platform API
  version: 1.0.0
  description: API for managing users, interviews, and reports for the AI Interviewer Platform.
servers:
  - url: /api/v1
paths:
  /auth/login:
    post:
      tags:
        - Authentication
      summary: Authenticate a user
      operationId: loginUser
      requestBody:

```

```
    required: true
  content:
    application/json:
      schema:
        type: object
        required: [email, password]
        properties:
          email: { type: string, format: email }
          password: { type: string, format: password }
  responses:
    '200':
      description: Authentication successful
      content:
        application/json:
          schema:
            type: object
            properties:
              accessToken: { type: string, description: "JWT for subsequent requests." }
    '401':
      description: Invalid credentials
/interviews:
  post:
    tags:
      - Interviews
    summary: Schedule a new interview
    operationId: scheduleInterview
    security:
      - bearerAuth:
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [candidateEmail, scheduledAt]
            properties:
              candidateEmail: { type: string, format: email }
              scheduledAt: { type: string, format: date-time }
              notes: { type: string }
    responses:
      '201':
        description: Interview scheduled successfully
        content:
```



```
    application/json:
      schema:
        $ref: '#/components/schemas/Interview'
  '400':
    description: Invalid input data
  '403':
    description: User does not have permission
/interviews/{interviewId}:
get:
  tags:
    - Interviews
  summary: Get interview details
  operationId: getInterviewById
  security:
    - bearerAuth:
  parameters:
    - name: interviewId
      in: path
      required: true
      schema: { type: string, format: uuid }
  responses:
    '200':
      description: Successful retrieval
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Interview'
    '404':
      description: Interview not found
/interviews/{interviewId}/start:
post:
  tags:
    - Interviews
  summary: Start an interview session
  operationId: startInterview
  security:
    - bearerAuth:
  parameters:
    - name: interviewId
      in: path
      required: true
      schema: { type: string, format: uuid }
  responses:
```

'200':
description: Session started
content:
application/json:
schema:
type: object
properties:
 websocketUrl: { type: string, format: uri }
 sessionToken: { type: string, description: "Token to authenticate the WebSocket
connection." }
/interviews/{interviewId}/report:
get:
tags:
- Reports
summary: Get the final analysis report
operationId: getInterviewReport
security:
- bearerAuth:
parameters:
- name: interviewId
in: path
required: true
schema: { type: string, format: uuid }
responses:
'200':
description: Report retrieved successfully
content:
application/json:
schema:
 \$ref: '#/components/schemas/InterviewReport'
'202':
description: Report generation is still in progress
'404':
description: Interview or report not found

components:
securitySchemes:
bearerAuth:
type: http
scheme: bearer
bearerFormat: JWT
schemas:
Interview:

```

type: object
properties:
  id: { type: string, format: uuid }
  candidateId: { type: string, format: uuid }
  interviewerId: { type: string, format: uuid }
  status: { type: string, enum: [scheduled, in-progress, completed, canceled] }
  scheduledAt: { type: string, format: date-time }
  createdAt: { type: string, format: date-time }
InterviewReport:
  type: object
  properties:
    interviewId: { type: string, format: uuid }
    summary: { type: string }
    scores:
      type: object
      properties:
        codeQuality: { type: number, format: float }
        problemSolving: { type: number, format: float }
        communication: { type: number, format: float }
    codeAnalysis: { type: object, description: "Detailed analysis of code quality, complexity,
etc." }
    speechAnalysis: { type: object, description: "Detailed analysis of communication and
vocal patterns." }
    timelineEvents:
      type: array
      items:
        type: object
        properties:
          timestamp: { type: string, format: date-time }
          eventType: { type: string }
          details: { type: object }

```

WebSocket Message Contracts

All messages exchanged over the persistent WebSocket connection will follow a standardized JSON structure, containing a type field to identify the message's purpose and a payload field containing the relevant data. This consistent structure simplifies message handling logic on both the client and server.

Client-to-Server Messages

- **Document Change (CRDT Operation):** Sent whenever the user edits code in the Monaco editor.

JSON

```
{
  "type": "doc_change",
  "payload": {
    "op": {
      // The CRDT operation data structure
    }
  }
}
```

- **Code Execution Request:** Sent when the user clicks the "Run Code" button.

JSON

```
{
  "type": "code_execute_request",
  "payload": {
    "language": "python",
    "code": "def main():\n print('Hello, World!')"
  }
}
```

- **Proctoring Event:** Sent when the client detects a relevant browser event (e.g., user switches tabs).

JSON

```
{
  "type": "proctoring_event",
  "payload": {
    "event": "tab_hidden",
    "timestamp": "2025-10-26T10:30:00Z"
  }
}
```

- **Audio Chunk:** Raw audio data is sent as a binary WebSocket message, not JSON.

Server-to-Client Messages

- **Document Update Broadcast:** Broadcast to all clients in a session (except the originator) when a doc_change is received.

JSON

```
{
```

```

"type": "doc_update",
"payload": {
  "sourceClientId": "client_id_of_originator",
  "op": {
    // The CRDT operation to apply
  }
}
}

```

- **Code Execution Result:** Sent to the originating client after their code has been executed.

JSON

```

{
  "type": "execution_result",
  "payload": {
    "stdout": "Hello, World!\n",
    "stderr": "",
    "executionTimeMs": 52
  }
}

```

- **Presence State Update:** Sent to all clients in a session when a user joins or leaves.

JSON

```

{
  "type": "presence_update",
  "payload": {
    "userId": "user_guid_xyz",
    "status": "joined"
  }
}

```

Authentication and Authorization Strategy

A robust security model is enforced at the API Gateway, acting as the primary line of defense and ensuring that no unauthenticated or unauthorized requests can reach the internal microservices.

- **Authentication:** The platform employs the **JSON Web Token (JWT)** standard.¹⁶ Upon a successful login via the POST /auth/login endpoint, the Node.js User & Auth Service generates a cryptographically signed JWT. This token contains a payload with essential claims, including the user's ID (sub), their role (e.g., interviewer), and an expiration timestamp (exp). For all subsequent requests to protected REST endpoints, the client

must include this JWT in the Authorization header using the Bearer scheme. The API Gateway is configured to intercept every incoming request, validate the JWT's signature and expiration, and reject any invalid requests with a 401 Unauthorized status before they are forwarded downstream.¹⁷

- **Authorization:** Authorization is implemented using **Role-Based Access Control (RBAC)**. The user's role is encoded as a claim within the JWT payload. The API Gateway, and as a secondary defense, the individual microservices themselves, will enforce access control rules based on this role. For example, a request to GET /interviews/{id}/report will be checked for the presence of the interviewer role in the token's claims. If the role is missing or incorrect, the request will be rejected with a 403 Forbidden status. This layered approach ensures that users can only access the resources and perform the actions appropriate for their designated level of permission.

4-Week Implementation Roadmap

This section outlines a concrete, four-week implementation roadmap designed to deliver a Minimum Viable Product (MVP) of the AI Interviewer Platform. The plan is structured to prioritize foundational work and enable parallel development streams across DevOps, Backend (Node.js and Python), and Frontend teams. The goal of this initial sprint is to establish a functional end-to-end slice of the platform, from user authentication and interview setup to real-time collaborative coding, basic code execution, and the generation of a preliminary analysis report. This phased approach provides clear weekly milestones for tracking progress and ensures that the most critical architectural components are validated early in the development lifecycle.

Week	Stream	Key Deliverables
1	DevOps	Establish Git monorepo and foundational CI/CD pipeline (build, test, containerize). Provision core cloud infrastructure (Kubernetes cluster, managed Kafka, PostgreSQL, Redis) using Terraform.
	Backend (Node.js)	Implement User & Auth service with JWT-based login (POST /auth/login). Implement basic Interview Orchestration service (POST /interviews). Set up NestJS project structure with Prisma for database access.
	Backend (Python)	Set up FastAPI project

		structures. Implement a basic Event Ingestion service that accepts events via HTTP and publishes them to a Kafka topic.
	Frontend	Initialize React application with Redux Toolkit for state management. Implement the login page and a basic dashboard to list scheduled interviews.
2	DevOps	Configure the API Gateway (e.g., Kong) for request routing and JWT validation. Deploy the initial observability stack (Prometheus for metrics, Loki for logs) to the cluster.
	Backend (Node.js)	Implement the Real-Time Collaboration service using a NestJS WebSocket Gateway. Establish basic text synchronization for the editor (without full CRDTs).
	Backend (Python)	Implement the Code Execution service to run Python code within a secure Docker sandbox. The service should publish execution results to a Kafka topic.
	Frontend	Integrate the Monaco Editor component into the interview view. Connect to the WebSocket server for real-time text updates. Implement the "Run Code" functionality to send code to the backend and display the output.
3	DevOps	Harden the CI/CD pipeline with automated security scanning (e.g., static analysis, container vulnerability scanning). Refine

		Kubernetes deployment manifests and scaling configurations (HPA).
	Backend (Node.js)	Integrate a CRDT library to enable conflict-free collaborative editing. Ensure the Interview Orchestration service correctly persists interview state changes (e.g., in-progress, completed) to PostgreSQL.
	Backend (Python)	Develop the first version of the Code Quality & Logic Agent. This agent will consume code_execution_result events from Kafka, run a simple linter (e.g., Pylint) on the code, and publish a structured insight event to the analysis-results topic.
	Frontend	Implement CRDT logic on the client side to manage concurrent edits in the Monaco editor, ensuring a smooth collaborative experience.
4	All Teams	Focus on Integration, End-to-End (E2E) Testing, and MVP Polish.
	Backend (Node.js)	Implement the Results & Reporting service. This service will consume the structured insights from the analysis-results Kafka topic, aggregate them into a simple report format, and persist the final report to PostgreSQL. Expose the GET /interviews/{id}/report endpoint.
	Backend (Python)	Implement the ingestion of proctoring events (e.g., tab

		switching) from the client. Enhance the AI agent to include this event in its analysis timeline.
	Frontend	Implement client-side detection of tab switching using the Page Visibility API and send the corresponding proctoring event over the WebSocket. Develop the report view to fetch and display the basic analysis report for a completed interview.

Conclusion and Strategic Enhancements

The architecture detailed in this document provides a robust, scalable, and innovative foundation for the AI-powered live interview platform. By integrating a specific three-tier technology stack (React, Node.js, Python) with the original microservices vision, this blueprint offers a clear and actionable path to implementation. The strategic adoption of an event-driven paradigm with Apache Kafka as its backbone ensures exceptional resilience and creates a durable data asset that future-proofs the platform for continuous AI innovation.¹ The dual-database strategy effectively balances the need for real-time performance with long-term data integrity, while the comprehensive security posture, aligned with the OWASP Top 10 for LLMs, addresses modern threats head-on.

The platform's core strategic advantage lies in its multi-agent AI system, orchestrated with LangChain. This approach moves beyond simplistic, single-model analysis to provide a multi-modal, synthesized assessment of a candidate's abilities, fusing signals from code, speech, and language to generate high-confidence, actionable insights.¹ This intelligence layer, combined with a secure and seamless real-time collaborative environment, positions the platform as a definitive leader in the field of technical talent assessment.

While this blueprint describes a complete and production-ready system, it is also designed for evolution. The following initiatives represent a strategic roadmap for future development, building upon the strong architectural foundation established herein:

1. **Advanced AI Model Integration:** Continuously research and integrate more sophisticated AI models. This includes leveraging multi-modal LLMs that can process audio and text simultaneously, fine-tuning custom models for specific programming languages or problem domains, and exploring models for detecting more nuanced

behavioral cues.

2. **Data Warehousing and Trend Analysis:** As the platform accumulates a large dataset of interview results, establish a data pipeline to load the structured reports from PostgreSQL into a cloud data warehouse (e.g., BigQuery, Snowflake). This will enable long-term trend analysis, allowing the organization to identify patterns in hiring, benchmark candidate performance across roles, and validate the predictive accuracy of the AI assessments against on-the-job performance.
3. **Third-Party ATS Integration:** Develop a robust integration layer to connect the platform with leading Applicant Tracking Systems (ATS). This would allow for seamless workflow automation, such as automatically creating interview sessions from an ATS and pushing the final analysis report back into the candidate's profile in the ATS.
4. **Expansion of Proctoring Capabilities:** Enhance the proctoring features by integrating optional webcam analysis (with candidate consent) to detect signs of distraction, such as excessive eye movement away from the screen, providing another data stream for the AI analysis agents.
5. **Personalized Candidate Feedback:** Leverage the AI-generated insights to provide automated, constructive feedback to candidates after the interview. This would enhance the candidate experience and provide value even to those who are not hired, strengthening the company's employer brand.

By following this roadmap, the platform can extend its initial architectural strengths to build a comprehensive ecosystem that not only identifies top technical talent but also continuously learns and improves, delivering ever-increasing value to the hiring process.

Works cited

1. Interview Platform Architecture Enhancement Report.pdf
2. OWASP Top 10 LLM and GenAI | Snyk Learn, accessed on October 29, 2025, <https://learn.snyk.io/learning-paths/owasp-top-10-llm/>
3. Home - OWASP Gen AI Security Project, accessed on October 29, 2025, <https://genai.owasp.org/>
4. Learn Redux Toolkit for State Management in React - CodeParrot AI, accessed on October 29, 2025, <https://codeparrot.ai/blogs/learn-redux-toolkit-for-state-management-in-react>
5. How to Use Redux Toolkit with ReactJS - FullStack Labs, accessed on October 29, 2025, <https://www.fullstack.com/labs/resources/blog/using-redux-toolkit-with-reactjs>
6. How to Use Redux Toolkit to Manage State in Your React Application, accessed on October 29, 2025, <https://www.freecodecamp.org/news/use-redux-toolkit-to-manage-state-in-react-apps/>
7. Monaco Editor, accessed on October 29, 2025, <https://microsoft.github.io/monaco-editor/>
8. Harnessing the Power of Monaco Editor in React Development, accessed on October 29, 2025,

<https://www.dhiwise.com/post/monaco-editor-a-comprehensive-guide-for-react-developers>

9. monaco-editor/react - NPM, accessed on October 29, 2025, <https://www.npmjs.com/package/@monaco-editor/react>
10. Monaco Editor for React. - GitHub, accessed on October 29, 2025, <https://github.com/react-monaco-editor/react-monaco-editor>
11. Prisma | NestJS - A progressive Node.js framework - NestJS Docs, accessed on October 29, 2025, <https://docs.nestjs.com/recipes/prisma>
12. Build Complete REST API Feature with Nest JS (Using Prisma and Postgresql) from Scratch - Beginner-friendly - PART 1 - DEV Community, accessed on October 29, 2025, <https://dev.to/alfism1/build-complete-rest-api-feature-with-nest-js-using-prisma-and-postgresql-from-scratch-beginner-friendly-part-1-5b73>
13. Building a GraphQL API with NestJS and Prisma - Jelani Harris, accessed on October 29, 2025, <https://jelaniharris.com/blog/building-a-graphql-api-with-nestjs-and-prisma/>
14. Practical Guide to Integrating Prisma with NestJS for Seamless Development - Medium, accessed on October 29, 2025, <https://medium.com/@rishabhgupta7210012474/practical-guide-to-integrating-prisma-with-nestjs-for-seamless-development-9f91e83cc990>
15. Build a REST API with NestJS, Prisma, PostgreSQL and Swagger, accessed on October 29, 2025, <https://www.prisma.io/blog/nestjs-prisma-rest-api-7D056s1BmOL0>
16. Authentication | NestJS - A progressive Node.js framework, accessed on October 29, 2025, <https://docs.nestjs.com/security/authentication>
17. A Step-by-Step Guide to Implement JWT Authentication in NestJS using Passport | Medium, accessed on October 29, 2025, <https://medium.com/@camillefauchier/implementing-authentication-in-nestjs-using-passport-and-jwt-5a565aa521de>
18. WebSockets - FastAPI, accessed on October 29, 2025, <https://fastapi.tiangolo.com/advanced/websockets/>
19. (2025) Fastapi gRPC, Check the Beginner's Guide Here - Apidog, accessed on October 29, 2025, <https://apidog.com/blog/fastapi-grpc/>
20. A Comprehensive Resource on Use gRPC with FastAPI - MojoAuth, accessed on October 29, 2025, <https://mojoauth.com/grpc/use-grpc-with-fastapi>
21. FastAPIWebsocketTransport - Pipecat, accessed on October 29, 2025, <https://docs.pipecat.ai/server/services/transport/fastapi-websocket>
22. SciencelO/whisper_streaming_web: Whisper Streaming ... - GitHub, accessed on October 29, 2025, https://github.com/SciencelO/whisper_streaming_web
23. Real-Time Voice Interactions with the WebSocket Audio Adapter - DEV Community, accessed on October 29, 2025, <https://dev.to/ag2ai/real-time-voice-interactions-with-the-websocket-audio-adapter-4keb>
24. gradion-ai/ipybox: A lightweight and secure Python code execution sandbox based on IPython and Docker - GitHub, accessed on October 29, 2025,

- <https://github.com/gradion-ai/ipybox>
25. Setting Up a Secure Python Sandbox for LLM Agents, accessed on October 29, 2025, <https://dida.do/blog/setting-up-a-secure-python-sandbox-for-llm-agents>
 26. Quickstart - Docs by LangChain, accessed on October 29, 2025, <https://docs.langchain.com/oss/python/langchain/quickstart>
 27. Medical large language models are vulnerable to data-poisoning attacks - PubMed, accessed on October 29, 2025, <https://pubmed.ncbi.nlm.nih.gov/39779928/>
 28. Integrate Monaco Editor - Sandpack - CodeSandbox, accessed on October 29, 2025, <https://sandpack.codesandbox.io/docs/guides/integrate-monaco-editor>
 29. LangChain | Your Enterprise AI needs Guardrails, accessed on October 29, 2025, <https://guardrailsai.com/docs/integrations/langchain>
 30. RunnableRails — NVIDIA NeMo Guardrails, accessed on October 29, 2025, <https://docs.nvidia.com/nemo/guardrails/latest/user-guides/langchain/runnable-rails.html>
 31. Adding guardrails to large language models. - GitHub, accessed on October 29, 2025, <https://github.com/guardrails-ai/guardrails>
 32. Guardrails - Docs by LangChain, accessed on October 29, 2025, <https://docs.langchain.com/oss/python/langchain/guardrails>
 33. How to implement LLM guardrails | OpenAI Cookbook, accessed on October 29, 2025, https://cookbook.openai.com/examples/how_to_use_guardrails
 34. Langchain Tutorial : Enhancing AI Safety with NeMo Guardrails | Input, Output, and with Examples - YouTube, accessed on October 29, 2025, https://www.youtube.com/watch?v=oDb_9Qwipow
 35. A Developer's Guide to Preventing Prompt Injection - Helicone, accessed on October 29, 2025, <https://www.helicone.ai/blog/preventing-prompt-injection>
 36. LLM Prompt Injection Prevention - OWASP Cheat Sheet Series, accessed on October 29, 2025, https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
 37. OWASP Top 10 LLM: How to test your Gen AI app in 2025 - Evidently AI, accessed on October 29, 2025, <https://www.evidentlyai.com/blog/owasp-top-10-llm>
 38. OWASP Top 10 2025 for LLM Applications: What's new? Risks, and Mitigation Techniques, accessed on October 29, 2025, <https://www.confident-ai.com/blog/owasp-top-10-2025-for-llm-applications-risks-and-mitigation-techniques>
 39. What are the OWASP Top 10 risks for LLMs? | Cloudflare, accessed on October 29, 2025, <https://www.cloudflare.com/learning/ai/owasp-top-10-risks-for-llms/>
 40. LLM01:2025 Prompt Injection - OWASP Gen AI Security Project, accessed on October 29, 2025, <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
 41. Protect Against Prompt Injection - IBM, accessed on October 29, 2025, <https://www.ibm.com/think/insights/prevent-prompt-injection>
 42. LLM02: Insecure Output Handling - OWASP GenAI Security Project, accessed on October 29, 2025, <https://genai.owasp.org/llmrisk2023-24/llm02-insecure-output-handling/>

43. Insecure output handling in LLMs | Tutorials & Examples - Snyk Learn, accessed on October 29, 2025, <https://learn.snyk.io/lesson/insecure-output-handling/>
44. What is LLM Insecure Output Handling? - Datavolo, accessed on October 29, 2025, <https://datavolo.io/2024/09/what-is-llm-insecure-output-handling/>
45. Introduction to LLM Insecure Output Handling - Cobalt.io, accessed on October 29, 2025, <https://www.cobalt.io/blog/llm-insecure-output-handling>
46. LLM05:2025 Improper Output Handling - OWASP Gen AI Security ..., accessed on October 29, 2025, <https://genai.owasp.org/llmrisk/llm052025-improper-output-handling/>
47. Insecure Output Handling - What is Bright DAST?, accessed on October 29, 2025, <https://docs.brightsec.com/docs/insecure-output-handling>
48. LLM04:2025 Data and Model Poisoning - OWASP Gen AI Security ..., accessed on October 29, 2025, <https://genai.owasp.org/llmrisk/llm042025-data-and-model-poisoning/>
49. Data Poisoning: a threat to LLM's Integrity and Security - RiskInsight, accessed on October 29, 2025, <https://www.riskinsight-wavestone.com/en/2024/10/data-poisoning-a-threat-to-llms-integrity-and-security/>
50. A Systematic Review of Poisoning Attacks Against Large Language Models - arXiv, accessed on October 29, 2025, <https://arxiv.org/html/2506.06518v1>
51. Data Poisoning: Current Trends and Recommended Defense Strategies - Wiz, accessed on October 29, 2025, <https://www.wiz.io/academy/data-poisoning>
52. Web API Design Best Practices - Azure Architecture Center ..., accessed on October 29, 2025, <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>
53. The WebSocket API (WebSockets) - Web APIs - MDN Web Docs, accessed on October 29, 2025, https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
54. System Design Basics: WebSockets - Medium, accessed on October 29, 2025, <https://medium.com/geekculture/system-design-basics-websockets-80aa2b5d5e52>
55. Writing WebSocket client applications - Web APIs | MDN, accessed on October 29, 2025, https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API/Writing_Web_Socket_client_applications
56. 10 Essentials When Creating an Open API Specification - Tyk.io, accessed on October 29, 2025, <https://tyk.io/blog/10-essentials-when-creating-an-open-api-specification/>
57. Tags Best Practices in OpenAPI - Speakeasy, accessed on October 29, 2025, <https://www.speakeasy.com/blog/tags-best-practices-in-openapi>
58. 14 Best Practices to Write OpenAPI for Better API Consumption - APIMatic, accessed on October 29, 2025, <https://www.apimatic.io/blog/2022/11/14-best-practices-to-write-openapi-for-better-api-consumption>